Juegos Belgranianos

Plataforma Digital del Campeonato Escolar

Product Roadmap & Plan de Aplicación Móvil

> **Documento de propuesta** · Versión 1.0 · Marzo 2026

---

1. Estado actual del sistema

✅ Fase 0 — Fundación

Feature	Estado
Proyecto Next.js 16 + TypeScript + Tailwind	✅
Supabase Auth (login, roles admin/operator)	✅
Schema PostgreSQL (editions, disciplines, teams, groups, matches, standings view)	✅
RLS policies + función `get_my_role()` anti-recursión	✅
Middleware de protección de rutas `/admin` y `/operator`	✅
Deploy en Vercel + Supabase free tier	✅

✅ Fase 1 — Admin Core

Feature	Estado
CRUD de ediciones del campeonato (nombre, año, fechas, estado)	✅
CRUD de disciplinas con parámetros de scheduling (canchas, duración, horario, intervalo)	✅
Formato de partidos por disciplina: solo ida / ida y vuelta	✅
CRUD de equipos registrados una sola vez con selección de disciplinas (many-to-many)	✅

Campo Grado/Año por equipo para partidos inter-grupos	✅
CRUD de grupos y asignación de equipos por grupo/disciplina	✅

✅ Fase 2 — Motor de Scheduling

Feature	Estado
Algoritmo round-robin (método círculo/polígono) para partidos intra-grupo	✅
Partidos inter-grupos por grado (`enable_cross_group`)	✅
Generación de slots disponibles por disciplina (horario, canchas, intervalo)	✅
Filtro de días de la semana habilitados para jugar	✅
Fechas bloqueadas por edición (feriados, descanso, otros)	✅
Motor greedy de asignación partido → slot	✅
Detección de conflictos (mismo equipo + misma hora + mismo género)	✅
Vista previa del calendario generado (por Jornada / Fecha / Disciplina)	✅
Confirmación del calendario → persistencia en base de datos	✅
Suspensión de partidos (scheduled → postponed)	✅
Reprogramación de partidos suspendidos (nueva fecha/hora/cancha)	✅

✅ Fase 3 — Resultados + Realtime

Feature	Estado
Layout del operador con auth server-side	✅
Dashboard del operador: partidos de hoy agrupados por horario	✅
Página de gestión de partido: Iniciar / Finalizar / botones +/- de score	✅
Campo de notas por partido	✅
Reapertura de partido finalizado para corrección de resultado	✅
Vista pública `/resultados` : partidos de hoy con scores en tiempo real	✅
Actualización automática vía Supabase Realtime	✅

Polling de 20s como fallback (cuando Realtime no está habilitado)	✅
Tabla de posiciones por grupo/disciplina con filtro	✅
Soporte de múltiples disciplinas y géneros en posiciones	✅

⏳ Fase 4 — Bracket Eliminatorio *(pendiente)*

Feature	Estado
Generación automática del bracket desde posiciones finales	⏳
Visualización del cuadro eliminatorio	⏳
Gestión de partidos de semifinal y final	⏳
Propagación automática de ganadores a la siguiente ronda	⏳

⏳ Fase 5 — Polish + Producción *(pendiente)*

Feature	Estado
Vista pública: fixture completo (todos los partidos, no solo hoy)	⏳
Vista pública: resultados históricos por fecha	⏳
Manejo de errores y estados de carga robustos	⏳
Optimización de queries (índices, paginación)	⏳
Hardening de RLS policies	⏳

---

2. Funcionalidades sugeridas de alto valor

Estas funciones no estaban en el plan original pero aportarían valor real al campeonato.

2.1 Goleadores / Máximos anotadores

Permitir al operador registrar qué jugador marcó cada gol. Requiere:

- Tabla `players (id, team_id, name, jersey_number)`
- Tabla `match_events (id, match_id, player_id, team_id, event_type, minute)`

- Vista pública: tabla de goleadores por disciplina
- **Valor**: los alumnos seguirían sus propias estadísticas

2.2 Notificaciones push

Enviar notificación cuando un partido inicia o finaliza.

- Supabase Edge Functions + Web Push API (PWA) o Expo Notifications (app móvil)
- **Valor**: padres y alumnos reciben el resultado sin abrir la app

2.3 Galería de fotos por partido / disciplina

- Subida de imágenes via Supabase Storage
- Galería pública en `/resultados`
- **Valor**: cobertura del evento, recuerdo para los alumnos

2.4 Historial de ediciones

- Vista pública de ediciones pasadas con campeones por disciplina
- **Valor**: continuidad año a año, orgullo institucional

2.5 Compartir resultado individual

- Botón "Compartir" en cada tarjeta de resultado
- Genera imagen con el score estilo "story" usando `html2canvas` o Supabase Edge Function
- **Valor**: viralidad orgánica en WhatsApp/Instagram del colegio

2.6 Panel de TV / Modo pantalla completa

- Vista `/tv` sin navegación, rotación automática entre disciplinas en vivo
- Diseñada para proyectar en pantallas en el colegio durante el evento
- **Valor**: experiencia presencial profesional

2.7 Estadísticas del campeonato

- Dashboard público: total de partidos jugados, goles, equipos participantes
- Gráficos de posiciones a lo largo del tiempo
- **Valor**: engagement y expectativa

2.8 Sistema de comentarios / reacciones por partido

- Reacciones emoji (⚽ 🔥 👏) en tiempo real vía Supabase Realtime

- **Valor**: participación de la comunidad durante el evento

---

3. Plan para aplicación móvil

3.1 Evaluación de opciones

Opción	Pros	Contras	Esfuerzo
PWA (Progressive Web App)	Cero código nuevo, instalable desde browser, comparte todo el código actual	No aparece en App Store/Play Store, sin notificaciones nativas en iOS	Bajo
Capacitor (wrapper nativo)	Reutiliza 100% del código Next.js, acceso a APIs nativas, distribución en stores	Rendimiento levemente inferior a nativo, WebView	Medio
React Native + Expo	Rendimiento nativo real, UI nativa, notificaciones push nativas	Reescribir toda la UI, mantener 2 codebases	Alto
Expo + shared logic	Comparte types y Supabase client, UI nativa	Complejidad de monorepo	Alto

3.2 Recomendación: Dos fases

Fase A — PWA inmediata (1-2 días)

Convertir la web actual en instalable. Los usuarios la agregan al home screen como si fuera una app.

Implementación:

```
src/app/
  manifest.json          ← nombre, íconos, colores, pantalla completa
  icons/                 ← íconos 192x192 y 512x512
next.config.ts           ← headers para service worker
public/sw.js             ← service worker básico (cache offline)
```

Qué se logra:

- Ícono en pantalla de inicio (Android e iOS)
- Pantalla completa sin barra del browser
- Carga offline básica (última data cacheada)
- Sin pasar por App Store — distribución por link/QR

Fase B — App React Native con Expo (proyecto separado)

Solo si se necesita: notificaciones push, distribución en stores, o UX nativa más pulida.

Stack recomendado:

```
belgraneanos-mobile/         ← repositorio separado
  src/
    lib/supabase/client.ts   ← mismo patrón, misma URL/key
    types/database.ts        ← copiado/compartido del proyecto web
    screens/
      ResultadosScreen.tsx
      OperatorDashboardScreen.tsx
      MatchDetailScreen.tsx
      StandingsScreen.tsx
  app.json                   ← config Expo
```

Lo que se reutiliza:

- Backend completo (Supabase — sin cambios)
- Tipos TypeScript (`database.ts`)
- Lógica de negocio (queries, Realtime)

Lo que se reescribe:

- Componentes UI (React Native Paper o NativeWind)
- Navegación (Expo Router o React Navigation)

3.3 Plan detallado — Fase A: PWA

Paso 1 — Manifest

Crear `src/app/manifest.json` :

```
{
  "name": "Juegos Belgranianos",
  "short_name": "Belgranianos",
  "start_url": "/resultados",
  "display": "standalone",
  "background_color": "#1e3a5f",
  "theme_color": "#1e3a5f",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png" }
```

```
  ]
}
```

Paso 2 — Enlace en el layout de la página pública

```
// src/app/layout.tsx
<link rel="manifest" href="/manifest.json" />
<meta name="theme-color" content="#1e3a5f" />
<meta name="apple-mobile-web-app-capable" content="yes" />
<meta name="apple-mobile-web-app-status-bar-style" content="default" />
```

Paso 3 — Service Worker básico

Para cachear la shell y funcionar offline.

Resultado esperado:

- Android: Chrome muestra "Agregar a pantalla de inicio" automáticamente
- iOS: Safari → Compartir → Agregar a pantalla de inicio
- Al abrir: pantalla completa, ícono propio, sin barra del browser

---

3.4 Plan detallado — Fase B: App Expo

Semana 1 — Resultados públicos

- Pantalla Home con partidos de hoy y scores en vivo
- Suscripción Realtime con `@supabase/supabase-js`
- Pantalla de Posiciones por disciplina/grupo
- Navegación por tabs (React Navigation Bottom Tabs)

Semana 2 — Operador

- Login con Supabase Auth
- Dashboard con partidos del día
- Pantalla de gestión de partido (botones +/-, Iniciar/Finalizar)
- Notificaciones push con Expo Notifications

Semana 3 — Polish

- Íconos, splash screen, colores del colegio

- Build para Android (APK/AAB via EAS Build)
- Build para iOS (requiere cuenta de desarrollador Apple)
- Publicación en Play Store (Google Play — ~USD 25 único)

3.5 Distribución recomendada

Para un campeonato escolar, **la PWA es suficiente en la mayoría de los casos**:

```
QR code → link → /resultados → "Agregar a inicio"
```

Los padres y alumnos no necesitan ir al App Store. Se distribuye por WhatsApp en los grupos del colegio.

La app Expo tiene sentido si:

- Se quiere publicar en Play Store / App Store formalmente
- Se necesitan notificaciones push reales (especialmente en iOS)
- El campeonato crece y se convierte en una plataforma permanente del colegio

---

4. Priorización sugerida

Prioridad	Trabajo	Valor	Esfuerzo
🔴 Alta	Fase 4: Bracket eliminatorio	Alto	Medio
🔴 Alta	PWA (manifest + íconos)	Alto	Muy bajo
🟡 Media	Panel de TV / modo pantalla completa	Alto	Bajo
🟡 Media	Fixture completo en vista pública	Medio	Bajo
🟡 Media	Historial de ediciones	Medio	Bajo
🟢 Baja	Goleadores/anotadores	Alto	Medio-alto
🟢 Baja	App React Native (Expo)	Alto	Alto
🟢 Baja	Notificaciones push	Alto	Medio
🟢 Baja	Galería de fotos	Medio	Medio